

BDD — Gerenciador de Oficina (Fase 4)

Visão de Negócio

O sistema foi projetado para oferecer uma solução robusta e escalável para o gerenciamento de oficinas, focando em automação, segurança e eficiência operacional. Os principais pilares do sistema incluem:

Objetivo	Descrição
Segurança	Cadastro e autenticação segura de usuários com JWT.
Operação	Gerenciamento completo do ciclo de vida de ordens de serviço.
Estoque	Controle rigoroso de peças e insumos.
Escalabilidade	Processamento assíncrono e infraestrutura elástica no Kubernetes.
Qualidade	Monitoramento em tempo real e pipelines de CI/CD automatizados.

Feature 1 — Autenticação e Controle de Acesso

Como usuário do sistema

Quero me autenticar

Para acessar os recursos protegidos da API

Cenários de Teste

Cenário: Cadastro de usuário administrador

- **Dado** que não existe usuário com o email informado

- **Quando** o cliente enviar uma requisição POST para `/usuarios` com perfil `ADM`
- **Então** o sistema deve retornar status `201`
- **E** o usuário deve ser persistido no PostgreSQL
- **E** a senha deve estar criptografada
- **E** o usuário deve estar ativo

Cenário: Login com credenciais válidas

- **Dado** que o usuário está ativo
- **Quando** ele enviar email e senha corretos para `/usuarios/login`
- **Então** o sistema deve retornar status `200`
- **E** deve gerar um token JWT válido
- **E** o token deve permitir acesso aos endpoints protegidos

Cenário: Acesso sem autenticação

- **Dado** que o endpoint exige autenticação
- **Quando** uma requisição for enviada sem token
- **Então** o sistema deve retornar status `401`



Feature 2 — Gerenciamento de Ordens de Serviço

Como administrador

Quero controlar o ciclo de vida das ordens de serviço

Para acompanhar o fluxo da oficina

Cenários de Teste

Cenário: Criar nova ordem de serviço

- **Dado** que o usuário está autenticado como `ADM`
- **Quando** enviar uma requisição POST para `/ordens`
- **Então** o sistema deve persistir a ordem no banco

- **E** deve registrar o status inicial como `DIAGNOSTICO`
- **E** deve retornar status `201`

Cenário: Evoluir status da ordem

- **Dado** que existe uma ordem no status `DIAGNOSTICO`
- **Quando** o status for alterado para `EXECUCAO`
- **Então** o sistema deve registrar a alteração
- **E** o tempo entre status deve ser mensurado
- **E** o evento deve ser enviado para monitoramento

Cenário: Finalizar ordem de serviço

- **Dado** que a ordem está em `EXECUCAO`
- **Quando** o status for alterado para `FINALIZADO`
- **Então** o sistema deve registrar a data de finalização
- **E** deve disparar evento para notificação



Feature 3 — Controle de Estoque

Como administrador

Quero controlar o estoque

Para evitar indisponibilidade de peças

Cenários de Teste

Cenário: Consumo de peça disponível

- **Dado** que a peça possui quantidade maior que zero
- **Quando** uma ordem consumir a peça
- **Então** o estoque deve ser decrementado
- **E** a operação deve ser registrada em log estruturado

Cenário: Tentativa de consumo sem estoque

- **Dado** que a peça possui quantidade zero
 - **Quando** tentar registrar o consumo
 - **Então** o sistema deve bloquear a operação
 - **E** deve retornar erro de regra de negócio
 - **E** deve gerar alerta de estoque crítico
-

Feature 4 — Processamento Assíncrono (Lambda + RabbitMQ)

Como sistema

Quero processar eventos de forma desacoplada

Para garantir escalabilidade e resiliência

Cenários de Teste

Cenário: Enviar notificação ao finalizar ordem

- **Dado** que uma ordem foi finalizada
- **Quando** o evento for publicado na fila
- **Então** a Lambda de notificação deve ser acionada
- **E** o cliente deve receber a notificação

Cenário: Falha no processamento do evento

- **Dado** que a Lambda falhou ao processar o evento
 - **Quando** ocorrer erro interno
 - **Então** o erro deve ser registrado
 - **E** o evento deve ser reprocessado conforme política definida
-



Feature 5 — API Gateway e Segurança

Como administrador do sistema

Quero proteger a aplicação

Para evitar abuso e requisições indevidas

Cenários de Teste

Cenário: Validação de token via Lambda Authorizer

- **Dado** que a requisição contém um token JWT
- **Quando** o API Gateway acionar a Lambda de autenticação
- **Então** o token deve ser validado
- **E** o acesso deve ser permitido se válido

Cenário: Rate limit excedido

- **Dado** que o cliente ultrapassou o limite configurado
- **Quando** novas requisições forem enviadas
- **Então** o API Gateway deve retornar status 429



Feature 6 — Escalabilidade (Kubernetes + HPA)

Como sistema

Quero escalar automaticamente

Para suportar alta demanda

Cenários de Teste

Cenário: Aumento de carga

- **Dado** que o consumo de CPU ultrapassou o limite configurado
- **Quando** o HPA detectar o aumento
- **Então** novos pods devem ser criados automaticamente

Cenário: Redução de carga

- **Dado** que o consumo voltou ao normal
 - **Quando** o HPA recalcular a média
 - **Então** pods excedentes devem ser removidos
-



Feature 7 — Observabilidade (New Relic)

Como time de operações

Quero monitorar o sistema em tempo real

Para agir rapidamente em caso de falhas

Cenários de Teste

Cenário: Alta latência em endpoint

- **Dado** que o tempo médio de resposta aumentou
- **Quando** ultrapassar o limite configurado
- **Então** um alerta deve ser disparado

Cenário: Falha no healthcheck

- **Dado** que `/actuator/health` retornar erro
 - **Quando** o Kubernetes executar a probe
 - **Então** o pod deve ser reiniciado
 - **E** o incidente deve ser registrado
-



Feature 8 — Pipeline CI/CD

Como time de desenvolvimento

Quero pipeline automatizada

Para garantir qualidade e padronização

Cenários de Teste

Cenário: Commit na branch principal

- **Dado** que um commit foi realizado
- **Quando** o GitHub Actions iniciar
- **Então** o projeto deve ser compilado
- **E** os testes devem ser executados
- **E** a análise do SonarQube deve validar o quality gate
- **E** a imagem Docker deve ser publicada
- **E** o deploy no EKS deve ser realizado automaticamente